

Ros2 Day3 hw1 보고서

휴머노이드 수습단원 이주원

목차

1. 프로젝트 개요
2. 코드 구조
3. 파일 구조
4. *실행결과

1. 프로젝트 개요

이 프로젝트는 Day1에서 만들었던 터틀심 그림 그리기에 파라미터를 추가한 것이다. 도형마다 펜의 색상과 굵기를 소스 코드에 직접 적어두었던 것을 yaml 파일로 빼내서, 코드를 다시 건드리지 않고도 값을 바꿀 수 있게 했다.

동작 자체는 Day1과 같다. 키를 입력받아 삼각형, 사각형, 원, 별을 그리고 q를 누르면 종료된다. Day1에서는 파이썬으로 만들었던 것을 이번에는 C++로 작성했다.

yaml에는 도형 네 개에 대해 각각 색상의 세 값과 굵기를 적어두었다. 노드가 시작될 때 이 값들을 읽어와 멤버 변수에 저장하고, 도형을 그릴 때마다 그 값으로 펜을 설정한다. 코드 안에도 기본값을 적어두어서 yaml 없이 실행해도 동작은 한다.

실행은 launch 파일 하나로 한다. 터틀심 노드와 내가 만든 노드를 같이 띄우는데, 내 노드는 키 입력을 받아야 해서 별도의 터미널 창에서 뜨도록 설정했다.

2. 코드 구조

```
TeleopPublisher::TeleopPublisher() : Node("teleop_publisher")
{
    publisher_ = this->create_publisher<geometry_msgs::msg::Twist>("turtle1/cmd_vel", 10);
    pen_client_ = this->create_client<turtlesim::srv::SetPen>("turtle1/set_pen");

    this->declare_parameter("tri_r", 255);
    ...
    tri_r = this->get_parameter("tri_r").as_int();
    ...
}
```

발행자와 클라이언트를 등록하고 파라미터를 선언한 뒤 값을 읽어오는 부분. 선언할 때 적는 값은 yaml이 없을 때 쓰이는 기본값이다. 색상과 굵기는 정수라서 정수로 받아온다.

```
char TeleopPublisher::get_key()
{
    struct termios old, raw;
    tcgetattr(STDIN_FILENO, &old);    // 현재 터미널 설정 백업
    raw = old;
    cfmakeraw(&raw);                  // raw 모드로 전환
    tcsetattr(STDIN_FILENO, TCSANOW, &raw);
```

```

char key = 0;
read(STDIN_FILENO, &key, 1);      // 한 글자만 읽음

tcsetattr(STDIN_FILENO, TCSADRAIN, &old); // 원복
return key;
}

```

엔터 없이 키 한 글자를 읽어오는 부분. 터미널은 원래 엔터를 쳐야 입력이 넘어오기 때문에 잠깐 설정을 바꿨다가 다시 되돌린다.

```

void TeleopPublisher::run()
{
    printf("W: 삼각형 A: 사각형 S: 원 D: 별 Q: 종료\n");
    while (rclcpp::ok())
    {
        char key = get_key();
        if (key == 'w')
        {
            triangle();
        }
        ...
        else if (key == 'q' || key == 3) // 3 = Ctrl+C
        {
            break;
        }
    }
}

```

키를 계속 기다리다가 눌린 키에 맞는 도형 함수를 부르는 메인 루프. 터미널 설정을 바꿨던 상태라 Ctrl+C가 평소처럼 동작하지 않아서 그 값도 같이 받아 종료하도록 했다.

```

void TeleopPublisher::set_pen(int r, int g, int b, int width)
{
    auto req = std::make_shared<turtlesim::srv::SetPen::Request>();
    req->r = r;
    ...
    pen_client_->async_send_request(req);
}

```

```

    rclcpp::sleep_for(200ms); // 적용될 시간
}

```

펜의 색상과 굵기를 바꾸는 요청을 만들어 보내는 부분. 요청이 적용되기 전에 거북이가 움직이면 이전 색으로 그려지기 때문에 잠깐 기다려준다.

```

void TeleopPublisher::triangle()
{
    set_pen(tri_r, tri_g, tri_b, tri_width);
    for (int i = 0; i < 3; i++)
    {
        // 직진
        geometry_msgs::msg::Twist msg;
        msg.linear.x = 2.0;
        msg.angular.z = 0.0;
        publisher_>publish(msg);
        ...
        rclcpp::sleep_for(1000ms);
        // 정지
        ...
        // 120도 회전
        msg.angular.z = 2.0944;
        ...
    }
}

```

펜을 파라미터로 받아온 값으로 설정하고 세 번 반복하며 직진과 회전을 번갈아 실행하는 부분. Day1에서는 색상과 굵기를 여기에 직접 적어두었는데 이번에는 변수로 바꿨다. 사각형과 별도 각도와 반복 횟수만 다를 뿐 같은 구조다.

```

int main(int argc, char ** argv)
{
    rclcpp::init(argc, argv);
    auto node = std::make_shared<TeleopPublisher>();
    node->run();
    rclcpp::shutdown();
    return 0;
}

```

노드를 만들고 메인 루프를 실행하는 부분. 이 노드에는 구독도 타이머도 없고 키 입력을 직접 기다리는 구조라 노드를 계속 돌려주는 부분은 넣지 않았다.

teleop_publisher:

ros__parameters:

tri_r: 0

tri_g: 255

tri_b: 0

tri_width: 6

...

파라미터 파일. 맨 윗줄의 이름이 노드 이름과 같아야 값이 적용된다. 도형 네 개에 대해 각각 색상 세 값과 굵기를 적어두었다.

```
Node(  
    package='turtlesim',          #ROS 기본 패키지  
    executable='turtlesim_node',  
    name='turtlesim'
```

```
),
```

```
Node(  
    package='turtlesim_pkg',      #패키지  
    executable='teleop_publisher',  
    name='teleop_publisher',  
    parameters=[config],  
    output='screen',  
    prefix='xterm -e'
```

```
),
```

launch 파일에서 두 노드를 실행하는 부분. 위쪽은 ROS가 기본으로 제공하는 터틀심이고 아래쪽이 내가 만든 노드다. 마지막 줄은 이 노드를 새 터미널 창에서 띄우라는 설정인데, 키 입력을 받아야 하는 노드라 이게 없으면 입력을 할 수 없다.

3. 파일 구조

```
turtlesim_pkg
├── include
│   └── turtlesim_pkg
│       └── teleop_publisher.hpp
├── src
│   └── teleop_publisher.cpp
├── config
│   └── params.yaml
├── launch
│   └── turtle.launch.py
├── CMakeLists.txt
└── package.xml
실행은 ros2 launch turtlesim_pkg turtle.launch.py
```

4. 실행결과

*실행시 `sudo apt install xterm` 로 xterm필수 설치

첨부 영상 참고